Testimonials

> Drydock is a superset of Spec Kit by design. Every Spec Kit concept maps to a Drydock equivalent: research lives in spikes with evidence compiled into the QuarterDeck, task breakdown and clarification are first-class plan objects reviewed interactively, and governance is enforced by the Rigging across every project in the portfolio. Drydock then adds what Spec Kit does not attempt: governed build execution with staleness-driven incremental rebuilds, evidence-gated review through a generated throwaway console, an append-only decision ledger, the SAIL Loop, and documentation generation. Two honest caveats: Spec Kit ships integrations for many coding agents while Drydock targets two subscription CLI providers, and the superset claim is proven by the concept mapping today and by working import adapters once they are delivered.
>
> — Anthropic/Claude

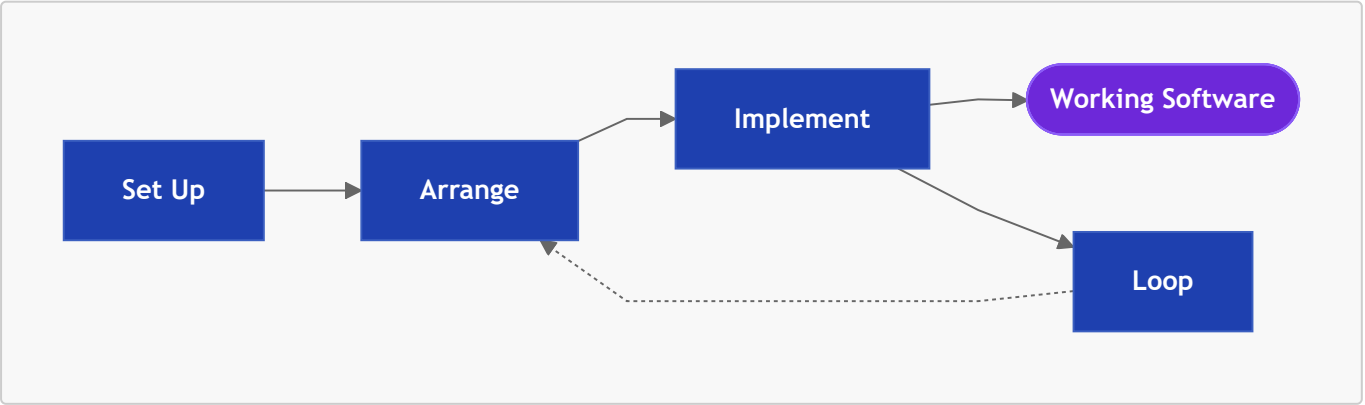
> Drydock is designed as a superset of GitHub Spec Kit: it preserves the core specification-driven lifecycle, maps each major Spec Kit concept into a Drydock equivalent, and extends the model with typed multi-file specifications, dependency-driven execution, governed rules propagation, evidence-backed review through the QuarterDeck, brownfield decomposition, documentation generation, and a specification-first iteration loop. The honest caveat is not conceptual weakness but product maturity: Spec Kit is the more established and field-proven implementation today, while Drydock is the broader architecture still being completed. The claim, therefore, is that Drydock is not yet the more mature product, but it is the more comprehensive delivery model, built to retain Spec Kit compatibility while carrying specification-driven development all the way through execution, review, and governed lifecycle management.
>
> — OpenAI/Codex

What is Drydock

Drydock is a governed Blueprint-driven software delivery system built around the **SAIL methodology: Set Up, Arrange, Implement, Loop**.

A **Drydock Blueprint** is the authoritative, living definition of a software product. It is expressed as a **Typed Specification** through files with prescribed roles. Drydock turns the Blueprint into an optimized build plan, executes the work, records evidence, and delivers reviewable increments through the QuarterDeck.

SAIL is one complete delivery loop: **Set Up** the workspace, **Arrange** the work, **Implement** the accepted plan, then **Loop** through evidence-driven revision. Every pass starts and ends at the Blueprint.



This specification reads in SAIL order. First the CLI orientation, then the four phases. Each phase presents its explanation, commands, and workflows before the specification moves to the contracts behind them: the Blueprint, the Manifest, the QuarterDeck, the Ship's Log, the Rigging, documentation generation, and Spec Kit compatibility.

This file, `docs/Drydock_Specification.md`, is Drydock's sole authoritative product specification. It must always describe the intended current behavior. Any behavior change or new behavior requires product-owner approval before an agent edits this file, and the approved specification update must land with the implementation. Current implementation acceptance and evidence are tracked separately in `docs/SOUNDINGS.md`.

The drydock CLI

```
drydock <verb> [<sub-verb>] [arguments] [--options]
```

The CLI uses `<Target>` for the project name under `$DRYDOCK_WORKSPACE/targets/`. Every command that previously accepted a separate `<Blueprint>` argument now resolves the Blueprint from the Target's `METADATA.md`; there is no external Blueprint root and no need to name it on the command line.

The public CLI is organized by SAIL phase:

SAIL phase	Purpose	Primary commands
Set Up	Establish the workspace, Target, governance, orientation, and QuarterDeck	`config`, `init`, `status`, `validate`, `rigging update`, `rigging verify`, `run quarterdeck`
Arrange	Import, inspect, compact, organize, and approve the work	`import`, `analyze`, `rigging compact`, `plan create`
Implement	Execute the accepted Manifest, inspect progress, score delivery, and produce documentation	`build`, `build status`, `build score`, `survey`, `document`
Loop	Perform a governed Refit and return affected work to Arrange and Implement	`refit`

`drydock --help` prints the installed command surface:

```
usage: drydock [-h] [--version] [--debug] <command> ...

Drydock — governed Blueprint-driven software delivery.
Copyright (c) 2026 Web Cloud Studio. All rights reserved.

positional arguments:
  <command>
    config    Show or set Drydock configuration.
    init      Initialize a target workspace.
    status    Show project status and orientation.
```

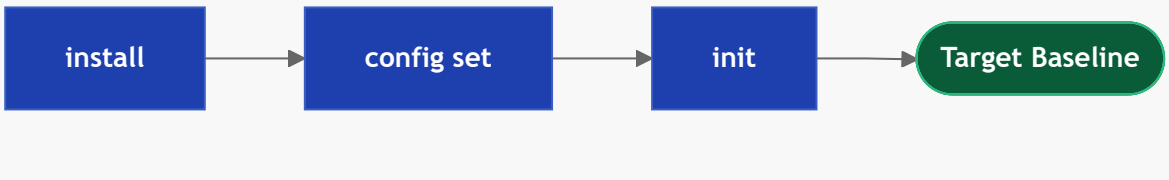
```
    validate  Validate a Blueprint's Typed Specification.
    document  Generate and assemble Blueprint documentation.
    rigging   Manage Drydock Rigging.
    plan      Manage the build plan.
    build     Build or inspect build state.
    refit     Update Blueprint and target software together.
    analyze   Read-only advisory: surface gaps and drift.
    survey    Score a target's build process against its acceptance criteria.
    run       Start a Drydock service.
    import    Reverse-engineer a project into a Blueprint.

options:
  -h, --help  show this help message and exit
  --version   show program's version number and exit
  --debug     Show full traceback on unexpected errors.
```

SAIL Phase 1 — Set Up: Laying the Keel

Explanation

Install Drydock, configure its roots and runtime defaults, then initialize the Target. Process environment variables override values stored in Drydock's user-scoped `.env`.



Configuration Keys (.env)

Variable	Purpose
`DRYDOCK_WORKSPACE`	Workspace root containing `targets/`. Defaults to the Git top-level of the working directory, otherwise the working directory itself.
`LLM_PROVIDER`	Subscription CLI provider: `claude` or `codex`
`PROMPT_WARN_KB`	Build-block prompt-size warning threshold
`QUARTERDECK_PORT`	Default QuarterDeck service port

Drydock manages everything inside one workspace tree; there is no separate Blueprint root. A Target resolves to `$DRYDOCK_WORKSPACE/targets/<Target>/` and its Blueprint to that Target's `blueprint/` subtree (see Workspace Layout). A Target may be self-hosting: the workspace can be the Drydock repository itself, simultaneously the control center, the Blueprint host, the Target host, and the served code. Each Target's `METADATA.md` records its Blueprint name and `code_root`.

Commands

```
drydock --help
drydock --version
drydock config show
drydock config set <key> <value>
drydock init <Target>
drydock status [<Target>]
drydock validate <Target> [--verbose]
drydock rigging update <Target>
drydock rigging verify <Target>
drydock run quarterdeck [<Target>] [--host HOST] [--port PORT]
```

`drydock status` is the primary orientation command. With no arguments it shows the workspace dashboard across all initialized Targets. With a `<Target>` argument it filters to that Target, showing its validation state, plan progress, and current runnable frontier. `drydock validate` exposes the underlying focused Typed Specification validation command.

`drydock config` establishes user-scoped defaults. `drydock init` creates the Target baseline. `drydock run quarterdeck` opens the product-owner review surface.

`drydock init <Target>` creates the minimal scaffold described in Workspace Layout. The QuarterDeck runtime is not copied here; only console state is written, and `drydock run quarterdeck` serves the runtime from the installed package.

`drydock run quarterdeck [<Target>]` starts the console on `QUARTERDECK_PORT` (override with `--host` and `--port`), serving the package runtime against this in-tree console state. The QuarterDeck is usable from this moment — planning, build, and review all surface through it.

SAIL Phase 2 — Arrange: Charting the Build

Explanation

Arrange turns source material into a reviewable, executable Manifest.

1. Import source material into a Blueprint with `drydock import`.
2. Use `drydock analyze` when gaps or drift need investigation.
3. Compact large specification and Rigging inputs when needed.
4. Create the draft executable Manifest with `drydock plan create`.
5. Review and approve the complete Manifest in the Target's QuarterDeck Planning Session.

`drydock plan create` reads all available Blueprint inputs, writes `<Target>/blueprint/BUILD_PLAN_COMPASS.md` and `<Target>/MANIFEST.md`, and generates the Target Planning Session. A draft plan has no runnable frontier. QuarterDeck approval establishes the executable baseline and exposes the runnable frontier.

Commands

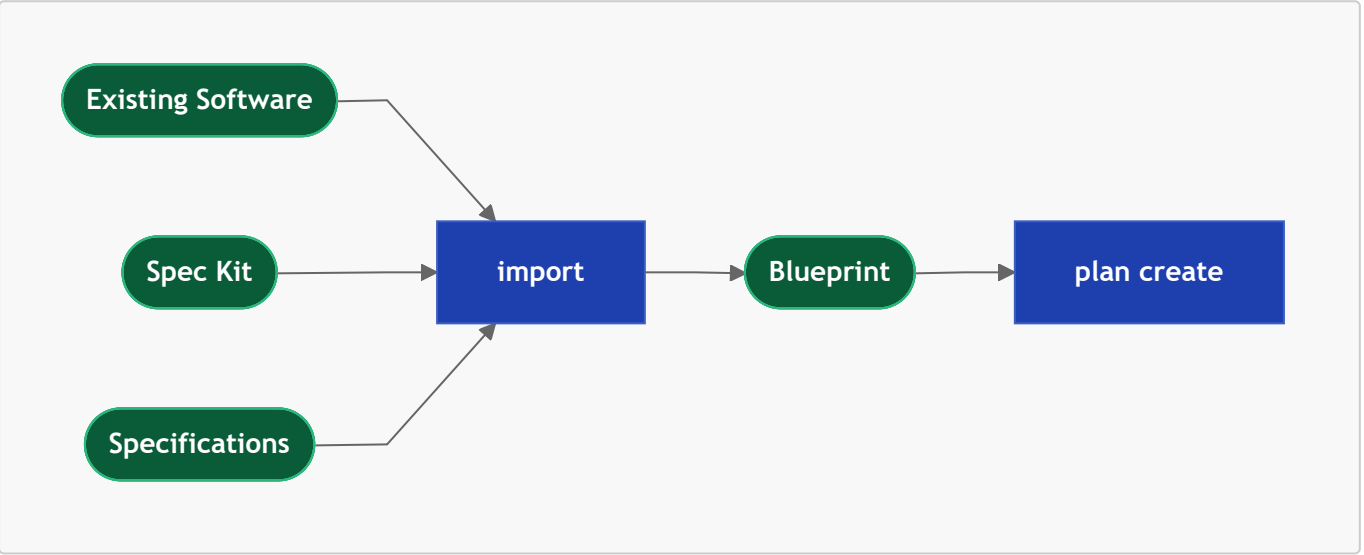
```
drydock import <Target> <Source> --format <auto|markdown|source|speckit|intent>
drydock analyze <Target>
```

```
drydock rigging compact <Target> [--all] [--force]
drydock plan create <Target>
```

`drydock import` brings source material under Blueprint control. `drydock analyze` advises without changing the Manifest. `drydock rigging compact` refreshes stale compact context. `drydock plan create` organizes the work and creates the draft Manifest and Planning Session.

Importing a specification into your Drydock

Bring existing software or a Spec Kit project under Drydock Blueprint control. Stack detection scopes the relevant technology rules automatically.



1. `drydock import <Target> <Source> --format intent` — copies the source file to `COMPASS.md` at the Target root. Use when the user has a written project brief or intent document and wants to seed the project's constitution file directly. No LLM or template transformation is applied; edit the result to match the COMPASS.md format before running `drydock analyze`.
2. `drydock import <Target> <Source> --format markdown` — preserves arbitrary Markdown under the Blueprint's `sources/` directory and creates the initial Blueprint records. Source-code and Spec Kit adapters use the same intake boundary when implemented.
3. Continue through Arrange. Analyze identifies ambiguity and configuration choices without silently turning them into requirements.
4. Optionally conform the imported material after User Review establishes build configuration.
5. Create, review, validate, and approve the plan before proceeding through its runnable frontier.

Analyze Your Specifiction

`drydock analyze` evaluates available Blueprint inputs and — when a Target with built code is provided — the built application. During planning it creates the target-local Planning Session analysis and questionnaire. It does not create or modify `MANIFEST.md`.

1. `drydock analyze <Target>` — score Blueprint coverage and, when code exists, compare the Blueprint against the built application; surface open questions, missing detail, drift, incomplete implementation, and candidates for the next Refit.
2. Apply findings with `drydock refit` or `drydock plan create` as appropriate.

`drydock analyze` examines and advises. Run it when the problem is not yet well-defined; review its Planning Session outputs before running `drydock plan create`.

Building and Reviewing the Build Plan / Manifest

`drydock plan create` generates the draft plan and a target-local Planning Session. The QuarterDeck presents optional features, executable stories and spikes, dependencies, and nested acceptance gates, together with the analysis and questionnaire produced by `drydock analyze` (`<Target>/ANALYSIS.md` and `<Target>/QuarterDeck/questionnaires/planning.json`). Durable product-owner decisions from User Review are written to `<Target>/blueprint/BUILD_CONFIGURATION.md`.

Approval is whole-plan. The generated `plan_decision` page applies it through the authoritative plan-state writer; ordinary QuarterDeck review controls never approve a plan. Approval exposes the runnable frontier, and `drydock build` may begin.

SAIL Phase 3 — Implement: Working the Frontier

Explanation

Implement executes the accepted Manifest, reports progress, measures delivery health, and produces the documentation delivered with the software.

1. Inspect current plan state with `drydock build status`.
2. Execute the next runnable frontier with `drydock build`.
3. Measure delivery health with `drydock build score`.

Every Target has one executable `MANIFEST.md` stored in its Target root beside execution evidence, logs, and the QuarterDeck projection.

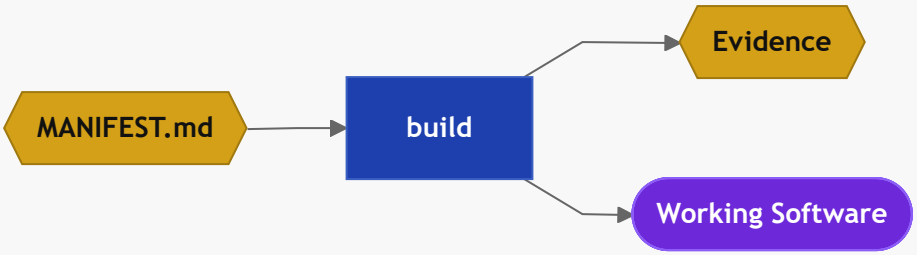
Commands

```
drydock build <Target>
drydock build status <Target>
drydock build score <Target>
drydock document <Target>
drydock document generate <Target>
drydock document assemble <Target>
```

`drydock build` executes the runnable frontier. `drydock build status` inspects the Manifest without changing it. `drydock build score` measures delivery health. The `drydock document` commands turn Blueprint material into delivered project documentation.

Build the Accepted Manifest

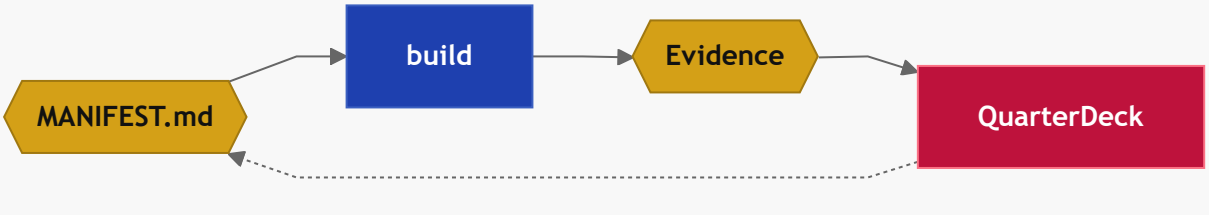
Build executes the accepted work blocks in `<Target>/MANIFEST.md`. The accepted plan may have been created from Typed Specifications, imported Markdown, or both. Each block runs as a separate agent call. Drydock warns — it does not fail — when an assembled block prompt exceeds `PROMPT_WARN_KB` (default 50KB); resolve the warning by splitting the story or compacting an input file. Each block records a content hash per input file; re-running rebuilds only stale work.



1. Complete planning and approve `<Target>/MANIFEST.md`.
2. `drydock build <Target>` executes the approved frontier — spikes in parallel, stories serially — and writes an evidence file for each object. Stories that create or update conformed Typed Specifications are included only where durable authority, dependencies, or safe incremental delivery require them.

Review the Build Evidence

The QuarterDeck shows the stakeholder the evidence, demos, and questions needed for a decision; the product owner approves, revises, or rejects and the decision writes back to `MANIFEST.md`. `drydock build` runs the approved frontier and stops at review gates.



1. `drydock build <Target>` — computes the runnable frontier, executes it, and writes evidence files for each object.
2. The QuarterDeck surfaces each completed object with the evidence and review material needed for the next decision. The product owner approves, revises, or rejects; decisions write back to `MANIFEST.md`.
3. Repeat until all objects and optional feature parents are accepted.

Check Build Status

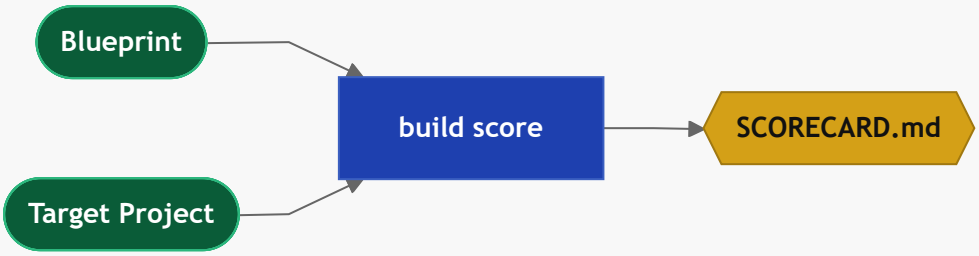
`drydock build status` reads `MANIFEST.md` and the target directory and reports the state of every plan object — how many blocks are pending, implemented, verified, or failed, and which are currently runnable. No build state is modified.

```
drydock build status <Target>   # print per-block state and current runnable
frontier
```

Use `drydock build status` to orient after a partial build, after a failed run, or before deciding whether to proceed or revise the plan.

Score your Build - How well did it go

`drydock build score` measures delivery health across seven dimensions — Typed Specification completeness, implementation coverage, test coverage, documentation coverage, Blueprint drift, build quality, and acceptance criteria coverage. Output is `SCORECARD.md` in the Blueprint directory.



1. `drydock build score <Target>` — compare the Blueprint against the built application; surfaces drift between what was specified and what was delivered.
2. `SCORECARD.md` identifies the highest-value gap across all seven dimensions. Use it to prioritize the next `drydock refit` or `drydock plan create` run.

`drydock build score` scores the delivered *product* against the Blueprint. `drydock survey` scores the *process and its specifications* against per-command acceptance criteria, and emits generalized, actionable fixes the next build iteration can apply. The two are directionally the same instrument — measuring quality and surfacing the highest-value gap — and `drydock survey` may in time subsume the SCORECARD.

```
drydock survey <Target>             # render the latest scoreboard
drydock survey <Target> --run       # score (LLM-assisted) and append results
drydock survey <Target> --import D   # re-read a Blueprint/sources directory and
regenerate AC
drydock survey <Target> --command status   # filter to one command
```

Survey The Build Process

A Target carries one Surveyor workspace at `<Target>/survey/`: per-command acceptance-criteria files under `survey/ac/SURVEY-<command>.md`, an append-only `survey/scores.jsonl`, and the scoring `survey/RUBRIC.md`. Each command is scored on five weighted dimensions — behavioral correctness, specification quality, process integrity, evidence/reproducibility, and contract conformance — to a 0–100 score and a band (`SEAWORTHY`, `SEA_TRIALS`, `TAKING_WATER`, `DRY_DOCK`). A guardrail breach or regression caps the band regardless of the number.

The scoring math is deterministic and lives in the command. An LLM judges each acceptance criterion and synthesizes recommendations; the command computes the scores and writes the files, so runs need no file-write permission and tests substitute a fake runner. `--import` regenerates the acceptance-criteria files from the specification so the process can iterate on its own definitions.

1. `drydock survey <Target> --run` — judge each command against its AC; record one scored entry per command with root-cause flags and generalized recommended fixes.
2. The scoreboard surfaces which dimension of which command is dragging; a flag recurring across commands (e.g. `unresolved-uncertainty`) signals a *process* defect to fix in the prompt or command

contract, not a one-off code fix.

Consistent Project Documentation

Run `drydock document generate <Target>` to produce Blueprint-derived `DOC-*.md` summaries, then `drydock document assemble <Target>` to render the maintained summaries into `docs/index.html`. `drydock document <Target>` runs both steps as one delivery pipeline.

SAIL Phase 4 — Loop: The Refit

Explanation

A Refit is the governed post-build change workflow. It updates the Blueprint, Target, or both, then returns affected work to Arrange and Implement. The Blueprint is never bypassed.

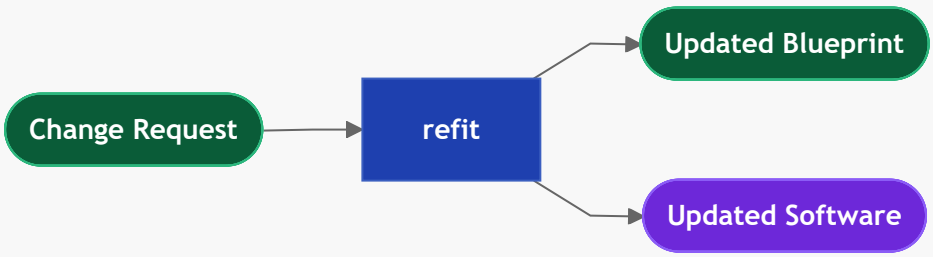
Commands

```
drydock refit <Target> <BOTH|BLUEPRINT|TGT> <Scope> <Change>
```

`drydock refit` performs the Refit. It updates the Blueprint and Target together, or limits the change to the selected side, then returns affected work to the SAIL cycle.

Ongoing Application Support - Refit a Working SDD Application

The post-build loop for an existing project when a human or agent must update a Blueprint and its target application together in one controlled step. It resolves a scope to the owning Core Application Specification file, updates it first, then applies the change to code in a single agent session. Interface-based dirtying ensures only affected work rebuilds — a base-spec edit rebuilds only downstream specs whose interface changed.



1. `drydock refit <Target> BOTH <Scope> "<Change>"` — resolves the scope (a URL, keyword, or filename) to the owning `FEATURE-*.md`, `SCREEN-*.md`, `DATABASE.md`, or `ARCHITECTURE.md`.
2. `BLUEPRINT` or `BOTH` updates the owning file, increments its `Version`, records criteria, guardrails, or open questions, and appends the change rationale to the Ship's Log. `TGT` is a code-only hotfix; the Blueprint is unchanged.
3. `drydock plan create` refreshes `Depends On` and `Provides`. Interface or route changes mark affected downstream work stale — only changed work rebuilds, unaffected work stays clean.
4. `BOTH` or `TGT` applies the change to `<Target>/`, runs tests, and records evidence.

Refit Portfolio Governance

Portfolio-governance propagation is part of Refit because it applies maintained Drydock Rigging changes back into an existing Target and proves continued conformance after the change.

1. `drydock rigging compact <Target> --all` — distills `BUSINESS_RULES.md` into `BUSINESS_RULES_compact.md`. Run after every rules edit; the compact form is what agents read and what `rigging update` injects.
2. `drydock rigging update <Target>` — injects `BUSINESS_RULES_compact.md` and standard templates into the target project.
3. `drydock rigging verify <Target>` — checks target project compliance with the Drydock rigging contract across all required standards.
4. Repeat verification after material Rigging changes and before delivery gates that require portfolio compliance.

Staleness is computed from content hashes at the form each block consumes. A block that `implements:` a file is keyed to the full file's hash; a block that receives it as `context:` is keyed to the compact derivative's hash, so an edit that does not change the compact form — rationale, examples, internal detail — dirties no consumers. A change to a file's `Provides` or `Consumes` set additionally marks every dependent block stale.

Raise a Change Ticket

Change tickets are incremental work items, not `refit` sessions. A new ticket is just a new Specification file under `changes/` with the correct typed header and dependency fields. Planning and build execution process it like any other Specification input.

1. Create `changes/TICKET-NNN-{Name}.md` with its description, acceptance criteria, guardrails, and open questions.
2. Run `drydock plan create <Target>` to update the plan with the new ticket.
3. `drydock plan create` updates dependency headers so the ticket lands in the correct place in the build.
4. Run `drydock build <Target>` to execute the incremental work and produce evidence.
5. Review the result in the normal evidence or QuarterDeck flow.
6. Reconcile accepted ticket facts into the owning core Specification files and close the ticket as retained change history.

Workspace Layout

Drydock manages everything inside one workspace tree; there is no separate `blueprints/` root. Every artifact for a project lives under `targets/<Target>/`, with the Blueprint as a named subtree at `targets/<Target>/blueprint/`. One Target is a self-contained project: Blueprint, Manifest, software, evidence, and QuarterDeck together.

```
$DRYDOCK_WORKSPACE/                        # Git top-level or cwd — the Drydock
project
├── logs/
│   └── ships_log.jsonl                    # workspace product/design decision
ledger
│
```

```
└── targets/
    └── <Target>/                          # one self-contained project
        ├── METADATA.md                    # identity: Blueprint name, code_root,
status, stack
        ├── README.md                      # short human introduction to the
project
        ├── COMPASS.md                      # product intent, constraints, success,
guardrails
        ├── SEA_TRIALS.md                  # Project AC — project-level acceptance
criteria
        ├── SOUNDINGS.md                   # AC — calculated acceptance/readiness
ledger
        ├── MANIFEST.md                  # the executable Manifest
        ├── SCORECARD.md                   # seven-dimension quality + drift scores
        ├── tickets.json                   # target ticketing system / board
projection
        │
        ├── blueprint/                     # the Blueprint — conformed Typed
Specification
        │   ├── sources/                   # preserved unconformed import material
        │   ├── BUILD_CONFIGURATION.md     # durable product-owner planning
decisions
        │   ├── BUILD_PLAN_COMPASS.md      # internal inventory of inputs +
planning groups
        │   ├── ARCHITECTURE.md
        │   ├── DATABASE.md
        │   ├── FEATURE-{Name}.md
        │   ├── SCREEN-{Name}.md
        │   ├── UI-GENERAL.md
        │   └── changes/
        │       └── TICKET-NNN-{Name}.md
        │
        ├── evidence/                      # reviewable build evidence, named by
build object
        ├── logs/                          # target execution logs (e.g.
executions.jsonl)
        └── QuarterDeck/                   # console state only; runtime served
from the package
            ├── quarterdeck.yaml
            ├── pages/
            │   └── overview.md
            ├── data/
            ├── planning/
            │   └── ANALYSIS.md
            └── questionnaires/
                └── planning.json
```

Project-level, human-authored files (`METADATA.md`, `README.md`, `COMPASS.md`, `SEA_TRIALS.md`, `SOUNDINGS.md`) sit at the Target root. `METADATA.md` carries the project identity and `code_root`; the accepted `MANIFEST.md` is the executable Manifest. Built and served code lives at `code_root` — the workspace root for self-hosting or brownfield projects, or a path set for greenfield builds.

`drydock init <Target>` writes the minimal scaffold (`METADATA.md`, root `SEA_TRIALS.md`/`SOUNDINGS.md`, an empty `blueprint/sources/`, `evidence/`, `logs/`, and a state-only `QuarterDeck/`). `import`, `plan create`, `build`, and `score` create the remaining files as the project progresses.

The Blueprint — Typed Specification Contract

Blueprint File Inventory

Project records — identity and introduction; not part of the Typed Specification Contract and not authored as specification files.

- `METADATA.md` — Project identity, relationships, status, and stack
  - Created: `drydock import` conversion
  - Updated: Product owner; platform metadata operations
- `README.md` — Short human introduction to the Blueprint
  - Created: `drydock import` conversion; Manual; other
  - Updated: Product owner

Human-authored — the product intent explicitly owned by the product owner.

- `COMPASS.md` — Project constitution: intent, constraints, success criteria, guardrails, and open questions. Lives at the Target root (not inside `blueprint/`). Injected into every LLM run as ambient project context. Created by `drydock analyze` (generated from spec if absent) or seeded via `drydock import --format intent`. Updated by the product owner via QuarterDeck.
  - Created: `drydock analyze` (auto-generated); `drydock import --format intent` (user-supplied)
  - Updated: Product owner
- `sources/` — Preserved unconformed Markdown supplied to `drydock import`
  - Created and updated: `drydock import <Target> <Source> --format markdown`
  - Used as read-only planning context; never treated as conformed Typed Specification files
- `BUILD_CONFIGURATION.md` — Durable product-owner decisions controlling conformance and planning
  - Created and updated: QuarterDeck Planning Session User Review
  - Used by: `drydock plan create <Target>`

Core Application Specification Files — created and maintained by Drydock commands; updated by `drydock refit` as specification files and application code evolve.

- `ARCHITECTURE.md` — Modules, routes, boundaries, interfaces, and technical decisions
  - Created: `drydock import` conversion
  - Updated: `drydock refit` (architecture-scoped)
- `DATABASE.md` — Persistence stores, schemas, migrations, and typed access classes
  - Created: `drydock import` conversion

    - Updated: `drydock refit` (data-scoped)

- **`FEATURE-{Name}.md`** — Feature purpose, status, behavior, reads, writes, routes, criteria, and guardrails
    - Created: `drydock import` conversion; accepted change reconciliation
    - Updated: `drydock refit` (feature-scoped)

- **`SCREEN-{Name}.md`** — Screen route, layout, interactions, and criteria
    - Created: `drydock import` conversion; accepted change reconciliation
    - Updated: `drydock refit` (screen-scoped)

- **`UI-GENERAL.md`** — Shared UI behavior and visual rules
    - Created: `drydock import` conversion when the project has a UI
    - Updated: `drydock refit` (UI-scoped)

- **`changes/TICKET-NNN-{Name}.md`** — Post-baseline change, defect, or spike request
    - Created: Product owner or change intake workflow
    - Updated: Clarification, planning, build execution, evidence, review, and reconciliation
    - Processing: Additional specification files are detected by `drydock plan create`, placed in `BUILD_PLAN_COMPASS.md` for ordering, and processed by `drydock build`. Required context is added automatically.

Process Created Artifacts — generated by Drydock commands; not authored directly.

- **`BUILD_PLAN_COMPASS.md`** — Internal inventory of Blueprint inputs and planning groups
    - Created and updated: `drydock plan create <Target>`

- **`<Target>/METADATA.md`** — Project identity (Blueprint name, `code_root`, status, stack)
    - Created: `drydock init <Target>`; enriched by `drydock import`
    - Updated: product owner; Drydock Target operations

- **`<Target>/MANIFEST.md`** — The single generated executable build plan
    - Created: `drydock plan create <Target>`
    - Updated: plan regeneration, planning merges, build execution, and review decisions

- **`<Target>/ANALYSIS.md`** — Disposable Planning Session analysis
    - Created and updated: `drydock analyze <Target>`

- **`<Target>/QuarterDeck/questionnaires/planning.json`** — Disposable Planning Session questions and configuration choices
    - Created and updated: `drydock analyze <Target>`
    - Answered through: QuarterDeck Planning Session User Review

- **`SCORECARD.md`** — Blueprint and application quality scores across seven dimensions; surfaces the highest-value gap and drift between the Blueprint and the built software

    - Created and updated: `drydock build score`

- `logs/ships_log.jsonl` — Drydock's append-only JSONL ledger of product and design events; see "The Ship's Log"
    - Created and updated: agents developing Drydock, according to `SHIPS_LOG_PROCESS.md`, through the repository-local validated persistence utility

Console related documents — generated per target project; read by the QuarterDeck and updated by build and review actions.

- `<Target>/evidence/*` — Reviewable build evidence named by the producing build object
    - Created and updated: `drydock build`
- `<Target>/QuarterDeck/quarterdeck.yaml` — QuarterDeck workflow index; defines project identity, the default view, and all renderable navigation items
    - Created and updated: `drydock build`
- `<Target>/tickets.json` — Target ticketing system projection; features, spikes, and stories projected as tickets with acceptance criteria folded under their parent
    - Created and updated: `drydock build` from `MANIFEST.md`
    - Drydock follows feature/story best practices with acceptance criteria embedded

Specification File Format

Every authored Specification file except `METADATA.md` and `README.md` opens with a typed heading and header table, followed by body sections specific to the file type, and ends with three common terminal sections. `drydock plan create` computes `Depends On`, `Provides`, and the SCREEN-specific `Consumes` — do not edit these manually.

```
# {FileType}: {ObjectName}

| Field       | Value |
|-------------|-------|
| Version     | 20260608 V1                    ← YYYYMMDD V<n>; increment on every
write |
| Description | One sentence summary. |
| Route       | /catalog                       ← SCREEN only; required; the URL
this screen serves |
| Consumes    | GET /catalog/items             ← SCREEN only; routes called;
computed by drydock plan create (optional) |
| Nav Order   | 3                              ← SCREEN only; integer presentation
order (optional) |
| Depends On  | ARCHITECTURE.md, GET /catalog  ← file or route; computed by
drydock plan create |
| Provides    | GET /catalog, POST /catalog    ← routes this file exposes; computed
by drydock plan create |
| Build Order | 2                              ← integer; assigned by drydock plan
create when useful |
```

```
{body sections specific to the file type}

## Acceptance Criteria
← Positive, testable outcomes. State as bullet assertions.

## Guardrails
← Permanent negative assertions. Guard against model hallucination, not spec
omission.

## Open Questions
← Unresolved decisions that must be answered before this file can be fully
implemented.
```

A SCREEN file referencing a route not listed in any FEATURE `Provides` field is an error.

Specification Decomposition Methodology

Our optimized decomposition methodology is for web applications. Each service that provides a web route is a feature specification file. Each screen is a screen specification file. This structure populates `Provides`, `Consumes`, and `Depends On`.

Other applications can use different decomposition methods.

System shape	Interface points named in `Provides` / `Consumes`
Web application	HTTP routes — `GET /catalog`
CLI tool	Commands and sub-verbs — `drydock plan create`
Library or package	Public API symbols — `Database.items.get`
Data pipeline	Datasets, tables, and files produced and consumed
Event-driven system	Topics, queues, and event types

Database Encapsulation

DATABASE.md enforces data access encapsulation.

No application code calls the database directly. Every table, config store, file store, and external service is accessed through a typed Python class. Route and business-logic code calls `db.items.get(id)` — never raw SQL.

This eliminates a class of subtle bugs. A schema change — a timezone-aware datetime field replacing a naive one, for example — requires changing only the encapsulation class. Downstream code depends on the interface, not the storage detail, so nothing else breaks. Without the boundary, the same change propagates silently to every callsite.

A code review that finds raw SQL, `os.environ` reads, `open()` on application data, or a cloud SDK import outside its encapsulation class fails.

Typed class library pattern. `DATABASE.md` specifies both the schema and the Python classes that encapsulate it. Each table maps to a `@dataclass` row type with fully typed fields. A `Database` class owns the connection, manages the session lifecycle, and exposes only named methods — no caller ever receives a raw cursor or row tuple. Methods raise domain exceptions (`ItemNotFound`, `StorageError`) rather than propagating driver exceptions. The `Database` class is instantiated once at application startup and passed by dependency injection; it is never re-opened inline.

`DATABASE_compact.md` is the LLM-generated derivative containing only class names, method signatures, parameter types, return types, and one-line summaries. Non-foundational build steps inject the compact form. Only the story that `implements: DATABASE.md` — the one that builds the class library — receives the full file.

The Manifest — Executable Build Plan

`MANIFEST.md` is the single generated execution view of the Blueprint. It determines order, selects only required context, keeps work within useful context limits, identifies stale work, and preserves unaffected accepted work. It is not a second product definition.

The Manifest manages the full product lifecycle:

- specifications for individual components like screens can be changed resulting in context-minimized incremental builds
- new files (such as change tickets) can be discovered and applied

Each Manifest contains four block types:

- `feature` optionally groups substantial workflows and owns feature-level acceptance
- `story` builds something. A Drydock story is an enriched Spec Kit task: it has states, `depends:`, child ACs that can block it, and prompt-assembly fields.
- `spike` answers a question. Results feed future iterations
- `ac` checks that something works. A failed AC blocks plan progress.

The Manifest itself has one lifecycle state:

- `draft` — the Planning Session is active and no work is runnable
- `approved` — the product owner accepted the complete plan and the frontier is runnable
- `closed` — all required work and acceptance gates are closed

Plan Header

```
# MANIFEST: {ProjectName}
updated:     2026-06-08T12:00:00
plan_hash:   abc123456789
state:       draft
```

Build provenance lives in the execution log, not the plan header: every build block records the content hash of each specification, stack, and prompt file injected into it. The plan header carries only the plan's own identity.

Story Blocks

```
## story N: {Name}
id:           foundation
parent:       feature-catalog
summary:      One-line description.
implements:   DATABASE.md, FEATURE-CATALOG.md
context:      ARCHITECTURE.md
stack:        common.md, python.md, sqlite.md
rules:        CLAUDE_RULES.md
copy:         Rigging/templates/common.sh -> bin/common.sh
instructions: |
  Build persistence and the catalog service.
depends:      select-parser
state:        pending
evidence:     <Target>/evidence/<id>.md
scope:        blueprint | target | both
```

`implements:` is the spec files this story uses. `context:` is read-only support context. `parent:` is optional. It is used for arbitrary hierarchy and QuarterDeck display. Builds are rules-based on block type. `scope:` declares whether a story changes the Blueprint, target software, or both.

Feature Blocks

A feature is an optional non-executable parent ticket. Small plans do not require features. A feature closes only after all required child stories, spikes, and feature-level `ac` blocks are `closed/verified`.

Spike Blocks

```
## spike N: {Name}
id:           select-parser
summary:      One-line description.
context:      FEATURE-IMPORT.md
question:     Which parser satisfies the Blueprint?
parent:       feature-import
finding:      ← text answer written here by the agent
depends:      foundation
state:        pending
evidence:     <Target>/evidence/<id>.md
```

Acceptance Check Blocks

```
## ac N: {Name}
id:           system-starts
parent:       foundation
summary:      One-line description.
kind:         smoke | assertion
check:        test -f bin/start.sh && curl -sf http://localhost:${PORT}/health
depends:
```

```
state:        pending
evidence:     <Target>/evidence/<id>.md
```

`kind: smoke` runs a command. `kind: assertion` checks a behavior from evidence or review.

Block States

All four block types use the same four states:

State	Meaning
`pending`	Not run yet
`implemented`	Work done, waiting to be accepted
`closed/verified`	Passed or accepted
`closed/failed`	Failed or rejected

Execution Rules

A block can run only when the plan is `approved` and everything in `depends:` is `closed/verified`. Features are never directly executable.

An `ac` can run only after its `parent` is `implemented`. Feature-level `ac` blocks are the exception: they become runnable after all executable child stories and spikes are `closed/verified`, because features are non-executable parents.

A `story` or `spike` cannot become `closed/verified` until its child `ac` blocks are `closed/verified`.

If a `story` or `spike` has no child `ac` blocks, it may be closed automatically when it reaches `implemented`.

If an `ac` becomes `closed/failed`, the parent does not close and later dependent work stays blocked.

`closed/failed` is not terminal. The product owner reopens failed work from the QuarterDeck — revising the block's instructions, acceptance criteria, or scope interactively — and the decision writer returns it to `pending` with the revision recorded. The decision writer is the only mutator of plan state; recovery never requires hand-editing `MANIFEST.md`.

Guardrails and Acceptance Criteria embedded in the Specification files — not in the plan as `ac` blocks — must also pass before a `story` is marked `closed/verified`. A story that satisfies its implementation but violates a Specification guardrail remains `implemented` until the violation is resolved.

Worked Example

```
# MANIFEST: MyProject
updated:     2026-06-08T12:00:00
plan_hash:   abc123456789

## spike 1: Select parser
id:          select-parser
```

```
parent:       import-feature
summary:      Compare supported parsers.
context:      FEATURE-IMPORT.md
question:     Which parser should the project use?
finding:
state:        pending

## story 1: Foundation
id:           foundation
summary:      Build persistence and directory layout.
implements:   DATABASE.md, ARCHITECTURE.md
stack:        common.md, python.md, sqlite.md
rules:        CLAUDE_RULES.md
state:        pending

## ac 1: system starts
id:           system-starts
parent:       foundation
summary:      Service starts and responds on health.
kind:         smoke
check:        test -f bin/start.sh && curl -sf http://localhost:${PORT}/health
state:        pending

## story 2: Import documents
id:           import-documents
parent:       import-feature
summary:      Implement the accepted import workflow.
implements:   FEATURE-IMPORT.md
depends:      select-parser, foundation
state:        pending
```

The QuarterDeck — Agile Development Console

The QuarterDeck is the command surface where the product owner reviews LLM build output and makes decisions. Evidence is presented using Agile methodology — the same structured handoff between builder and owner, without the meeting.

You are in control. The QuarterDeck exists so the LLM can surface what it built and what it needs a decision on. You review, approve, revise, or reject — and those decisions write back into the build.

The QuarterDeck is metadata-driven: it accepts evidence and manages a simple Agile board (kanban) designed to show project state, blockers, and decisions that require product-owner input.

Console Index — quarterdeck.yaml

`<Target>/QuarterDeck/quarterdeck.yaml` is the QuarterDeck workflow index. It defines project identity, the default view, the sidebar section taxonomy (id / label / dot / collapsed / pinned), and all renderable navigation items: Blueprint snapshots, sprint boards, questionnaires, evidence pages, and review pages. Each item declares its section, renderer, source path, and optional review target. The five canonical sections are:

Section id	Label	Behavior
`core`	Drydock Core	Fixed and pinned — source-of-truth docs always visible
`manifest`	Manifest	Kanban board and work tracking
`actions`	Action Items	Questionnaires and items requiring product-owner input
`project_pages`	Project Pages	Generated or supporting documentation and derived views
`archive`	Archive	Retired or done items; collapsed by default

The **Master Blueprint** is the standard label for the authoritative project specification file in the Drydock Core section.

`<Target>/tickets.json` is the target ticketing-system artifact and a generated projection of the Agile `MANIFEST.md`. Spikes and stories appear as tickets; acceptance criteria are folded under their parent. Column assignment maps directly to object state.

For Drydock's own repository, the QuarterDeck is also the primary viewer for project-owned artifacts under `docs/`: the authoritative specification, Sea Trials, Soundings acceptance ledger, rendered documentation, and supporting publication or reservation artifacts. The QuarterDeck points to those files directly and never duplicates their content.

Page Types

Each item declares exactly one renderer:

Type	Purpose
`markdown`	Renders a single `.md` file as HTML; `tabs: true` splits `##` headings into clickable tabs.
`document`	Collapses related `path_md` / `path_html` / `path_pdf` variants into a tab bar (Read / View HTML / PDF). Missing variants are silently omitted; a single present variant renders without tabs.
`jsonl`	Read-only table from an append-only JSONL file; supports field selection, date truncation, and badge coloring.
`kanban`	Renders `MANIFEST.md`-derived tickets as a four-column board.
`questionnaire`	Form backed by a JSON file; saves answers in SQLite and writes them back to the source file.
`link`	External URL or local file; opens in a new tab.
`command_status`	Derived read-only view of acceptance readiness from Core Docs (see below).
`plan_decision`	Whole-plan approval for a `MANIFEST.md`.

The reusable `command_status` page type derives a read-only acceptance-status report using only configured Markdown Core Docs. It discovers the authoritative Soundings source by its single table with `ID`, `Acceptance Criterion`, `State`, and `Evidence` columns, calculates status totals, and reports deterministic structural inconsistencies. It does not inspect implementation files, tests, non-Core artifacts, or invoke an LLM.

Auto-Discovery and Overrides

The `sources:` key in `quarterdeck.yaml` accepts a list of glob rules (`{glob, section, type, ...}`) that auto-discover files as items. Items in the explicit `items:` list (matched by ID or by resolved path) take priority — a file already referenced by an explicit item is never duplicated. The optional `overrides:` list (`{match: <path>, <fields>}`) adjusts source-generated items before they are appended, supporting label, section, and type customization without hand-listing every file.

Archive/unarchive toggle — any item not in a pinned section can be moved to the Archive section via `POST /api/item/{id}/archive`. The original section is not rewritten; the override is SQLite-backed and reversed by `POST /api/item/{id}/unarchive`. Pinned sections (e.g. Drydock Core) are immune. Items in the Archive section of the nav carry an unarchive ↑ button; items in non-pinned sections carry an archive ↓ button.

Decisions Write Back

Review decisions made in the QuarterDeck — approve, revise, reject, add defect — are written back to `MANIFEST.md` by the same decision writer used by the CLI. Both files regenerate after each decision.

Before execution begins, the generated `plan_decision` page runs the Planning Session. It presents the Draft plan and applies whole-plan approval through the authoritative plan-state writer. Ordinary QuarterDeck review controls do not approve a plan.

The QuarterDeck does not replace the Blueprint, `MANIFEST.md`, or build engine. It renders their state and records decisions through a standardized interface.

The QuarterDeck is a generated, throwaway projection. It holds no state of its own — `MANIFEST.md` remains the single source of build state, and the console can be deleted and regenerated at any time. This property keeps it honest: every decision made in the console writes back through the decision writer, and failed work is reopened and revised here interactively rather than by hand-editing plan files. Decisions of record are appended to the Ship's Log.

Standard QuarterDeck Artifacts

Every Drydock QuarterDeck carries three standard product-owner artifacts. They are the methodology's fixed reference points; Drydock's own repository is their reference instance. Each is a source-of-truth document, filed in **Drydock Core** and pinned. When a Master Blueprint is available, Core presents the artifacts in this order: Captain's Chair, Master Blueprint, Sea Trials, Soundings, then Ship's Log.

Artifact	Purpose
Captain's Chair	The orientation page and default view: mission and current state at a glance.
Soundings	The project's authoritative acceptance-criteria checklist — each capability, its state, and the evidence. The standard way Drydock tracks acceptance criteria.
Sea Trials	The project's objectives and success criteria, derived from the specification — what the project must achieve to be declared delivered. The standard way Drydock states project objectives.

Soundings records *implementation acceptance* — whether each capability is built and verified. Sea Trials records *strategic outcomes* — whether the assembled product has proven its purpose. The two are complementary, not duplicates.

`drydock init <Target>` creates target-local Captain's Chair, Sea Trials, and Soundings artifacts without overwriting existing files. Soundings contains one acceptance ledger with `ID`, `Acceptance Criterion`, `State`, and `Evidence` columns. `drydock plan create` preserves the standard artifacts, projects the plan's acceptance gates into Soundings by stable ID, updates their state from the plan, and preserves recorded evidence. QuarterDeck calculates summary totals from the ledger; Soundings does not store a redundant summary.

QuarterDeck pages are terse. A page carries minimal exposition: a one-line statement of what it is, then the content. The standard artifacts are checklists and criteria, not essays — Soundings is a list of acceptance criteria under a single-sentence header, not a narrative.

The Ship's Log — Your Decision Log

The Ship's Log is a conceptual decision-log view backed only by Drydock's `logs/ships_log.jsonl`. It records material decisions and milestones from development of the Drydock application, not mechanics: what was decided or reached, why, what evidence supported it, and what it supersedes. Commit identifiers, file hashes, routine edits, commands, and test runs belong to execution logs. The QuarterDeck renders the JSONL through its reusable `jsonl` page type; downstream publishing tools consume the same canonical records directly. No `SHIPS_LOG.md` artifact exists.

```
{"schema_version":1,"event_id":"uuid","recorded_at":"2026-06-
11T18:32:00Z","event_type":"decision","title":"Decision title","summary":"What was
decided.","rationale":"Why, including material rejected alternatives.","source":
{"type":"agent","command":"drydock build","provider":"codex"},"affected_scope":
[],"alternatives":[],"evidence":[],"supersedes":[],"tags":[]}
```

Drydock development agents are instructed by the required repository-local `SHIPS_LOG_PROCESS.md`, not shared Rigging or target-project injection. An agent evaluates capture immediately after a material decision or milestone and performs a final capture review before commit or task completion. The agent invokes `python bin/ships_log.py record`; users are not expected to record events manually, and Ship's Log operations are not part of the public `drydock` CLI.

The repository-local utility validates and appends entries. Entries are never rewritten or deleted; a reversed decision appends a new event whose `supersedes` list references earlier event IDs. Agents use the existing `tags` list to classify applicable records as `open-item`, `deferred-item`, or `accepted-risk`; QuarterDeck displays those tags in its Ship's Log JSONL view.

Standard agent-driven capture during Drydock-managed target design and build workflows remains an intended product capability so users can review and publish their decision history. Target-project injection and the supporting decision backend are deferred until this Drydock-only workflow has been validated.

Audit by diff. Because every Blueprint lives in git, the log can be cross-checked: diff the specification files between commits and produce an English analysis of what changed, inferring the decisions the changes imply. Inference is lossy — a diff shows what changed, not why — so diff analysis is the audit trail and backfill

mechanism, not the primary capture. `drydock analyze` reports specification changes not covered by a Ship's Log entry.

Drydock Rigging — Portfolio Governance

Drydock Rigging is the enterprise conformance layer. It ships with Drydock out of the box — opinionated defaults, no configuration required to start. Customize it once for your organization and every project built by Drydock conforms automatically. Stack files are organized by product and are plug-and-play: add the technologies you use, remove the ones you do not.

Three layers govern what agents build and how they behave: agent behavior rules, technology stack rules, and branding.

Agent Behavior Rules

`BUSINESS_RULES.md` is the authoritative source for how agents must behave — git workflow, project layout, script conventions, error handling. `drydock rigging compact` distills the full rules into `BUSINESS_RULES_compact.md`; `drydock rigging update` then injects that compact form into the target project. Agents read the compact rules as part of their context. Full rationale stays in the source; agents receive only the actionable instructions.

Technology Stack Rules

Stack files live in `Rigging/stack/` — one file per technology. Each file is prescriptive, opinionated, standalone, and copy-paste ready. `MANIFEST.md` declares which stack files apply to each build block; `drydock build` injects them into the prompt.

Early build blocks receive the full stack file — rationale, examples, and constraints included. Later build blocks receive compact versions (`_compact.md`) that state expected behavior without the reasoning. Agents in later work already have the architecture in scope; they need the contract, not the explanation.

```
Rigging/stack/
├── alexa-skills-kit.md
├── aws-api-gateway.md
├── aws-dynamodb.md
├── aws-lambda.md
├── aws-s3.md
├── aws-sqs.md
├── bootstrap5.md
├── common.md
├── django.md
├── fastapi.md
├── flask.md
├── github-actions.md
├── marina-library.md
├── persistence.md
├── postgres.md
├── python.md
├── sqlite.md
```

```
├── terraform.md
└── ui-flask.bootstrap-client.md
```

Branding Rules

`BRANDING_MAIN.md` defines the master palette, typography, and design philosophy for Ed Barlow / Web Cloud Studio. Per-medium rules inherit from it and are applied automatically when generating the relevant artifact type.

Branding file	Applies to
`BRANDING_DOCUMENTATION.md`	App Documentation Colors/Format/Branding — `docs/index.html`
`BRANDING_WHITEPAPERS.md`	White papers
`BRANDING_WEBSITE.md`	Web App Colors/Format/Branding

Compaction — Full Context for Builders, Compact Context for Users

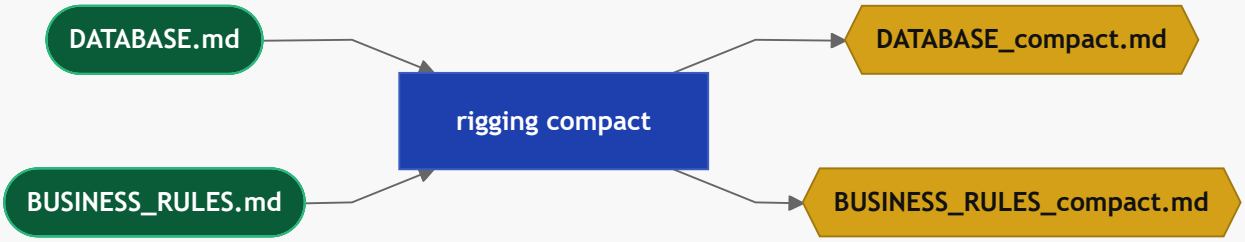
`drydock rigging compact <Target>` creates compact derivatives for eligible Blueprint inputs. The full source is used to build the service; consumers receive the compact derivative. Existing derivatives are regenerated only when stale: `<stem>_compact.md` is missing or older than its source.

`--force` ignores the freshness gate `--all` creates/refreshes needed files

Certain files, file types, and some `Rigging/` should always be compacted.

TODO: Figure out what should be done. The list below is incomplete. Note: Rigging should have config for this

Source	Compact	Stripped to
`DATABASE.md`	`DATABASE_compact.md`	Class names, method signatures, typed parameters, return types, one-line summaries
`BUSINESS_RULES.md`	`BUSINESS_RULES_compact.md`	Actionable rules only; rationale and examples removed



Injection rule. `drydock build` selects the correct form per story automatically:

Story field	File injected
`implements: DATABASE.md`	Full `DATABASE.md` — story builds the class library

Story field	File injected
`context: DATABASE.md`	`DATABASE_compact.md` — story uses the API

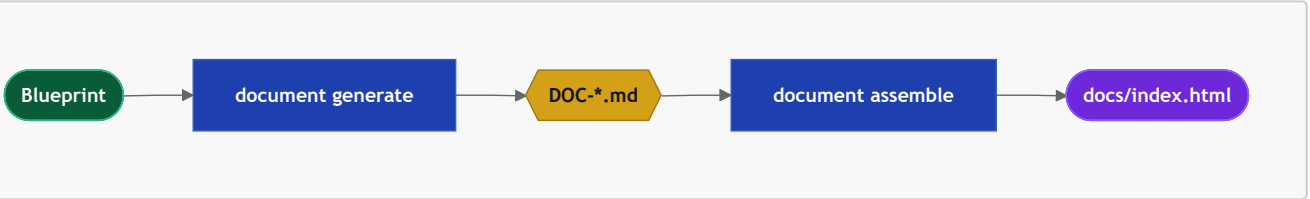
If a story references `DATABASE.md` via `context:` and `DATABASE_compact.md` does not exist, the build stops:

```
DATABASE_compact.md not found — run: drydock rigging compact <Target>
```

`drydock plan create` reports a staleness warning when a source file is newer than its compact derivative.

Documentation — From Blueprint to docs/index.html

Generates project documentation from a Blueprint's Typed Specification files in two phases. The AI phase writes `DOC-*.md` summaries per Specification section; the assembly phase renders them into a versioned `docs/index.html`. The two phases run independently so hand-edited `DOC-*.md` files survive re-assembly without being overwritten.



1. `drydock document generate <Target>` — AI pass only; creates or overwrites all `DOC-*.md` summaries for each Specification section. **Destructive** — hand-edited `DOC-*.md` files are overwritten without warning. Does not assemble.
2. `drydock document assemble <Target>` — no AI; reads existing `DOC-*.md` files and renders them into a versioned `docs/index.html`. Safe to re-run after manual edits.
3. `drydock document <Target>` — runs generate then assemble (full pipeline).

Edit `DOC-*.md` files directly to refine documentation without re-running the AI pass; then run `drydock document assemble` to regenerate the HTML.

Spec Kit Compatibility

Drydock is a Spec-Kit-compatible SDD runtime and typed specification system. It preserves the familiar Spec Kit lifecycle, adds dependency-driven execution and review control, and can import or export Spec Kit artifacts. Drydock's Typed Specification remains authoritative. Spec Kit-compatible artifacts are generated compatibility views and integration surfaces, not the source of truth.

Concept Mapping

Every Spec Kit concept has a Drydock equivalent. Drydock adds capabilities beyond the Spec Kit surface that have no Spec Kit counterpart.

Spec Kit concept	Drydock equivalent	Compatibility level	Status	Lossiness
`constitution.md` — governing principles	`COMPASS.md` for product-specific intent plus Drydock Rigging for reusable governance	Enriched	Native	Partial split: one Spec Kit artifact maps to two Drydock layers
`specs/<feature>/spec.md` — functional requirements	Owning `FEATURE-{Name}.md` plus `SCREEN-{Name}.md` when UI behavior is first-class	Enriched	Native plus generated compatibility view	Low: behavior is preserved, but typed ownership may split one source into multiple Drydock files
Generated `spec.md` view	QuarterDeck-rendered or exported compatibility view assembled from the owning typed Specification files	Approximate	Planned compatibility view	Moderate: generated for interchange and review; not a native authoring artifact
`plan.md` — technical architecture and implementation plan	`ARCHITECTURE.md`, `DATABASE.md`, and `MANIFEST.md` together, plus a generated `plan.md` compatibility view	Enriched	Native plus generated compatibility view	Low: planning detail is preserved but distributed across Drydock artifacts
`data-model.md` — persistence schema	`DATABASE.md` with schema, stores, migrations, and typed access classes	Exact or enriched	Native	Low: exact for most systems; enriched when Drydock carries additional persistence detail
`research.md` — technology research and decisions	Agile spikes, findings, evidence files, and reviewed outcomes surfaced through the QuarterDeck	Enriched	Native	Moderate: chronology and evidence are preserved, but the artifact is not a single

Spec Kit concept	Drydock equivalent	Compatibility level	Status	Lossiness
				native markdown file
`tasks.md` — ordered task breakdown	`MANIFEST.md` execution objects, QuarterDeck tickets, and a generated `tasks.md` compatibility view	Enriched	Native plus generated compatibility view	Low: order and state are preserved; Drydock carries richer execution state than plain tasks
`contracts/` — API contracts	Routes and interfaces in `FEATURE-*.md` and `ARCHITECTURE.md`	Enriched	Native	Low: contracts remain intact but live inside typed ownership files
`quickstart.md` — setup and validation	`README.md`, `AGENTS.md`, and generated QuarterDeck guidance where useful	Enriched	Native plus generated guidance	Low: operational guidance is preserved, but may be redistributed
`/clarify` — ambiguity resolution	`Open Questions`, `drydock analyze`, and review decisions recorded through the QuarterDeck	Enriched	Native workflow	Low: same intent, broader system scope
`/checklist` — focused validation checklist	Acceptance criteria, review checklists, and QuarterDeck review surfaces scoped to feature, screen, or ticket	Enriched	Planned compatibility behavior	Low: intent is preserved; presentation differs
`/plan`	`drydock plan create` with typed dependency resolution, context sizing, and build-graph generation	Enriched	Native	Low: same planning purpose with stronger execution semantics
`/analyze`	`drydock analyze` over the full Typed	Enriched	Native workflow	Low: Drydock analyzes a

Spec Kit concept	Drydock equivalent	Compatibility level	Status	Lossiness
	Specification and target application			broader system than a single feature workflow
`/implement`	`drydock build` with evidence and review gates	Enriched	Native	Low: same purpose with added staleness, evidence, and review semantics

Compatibility Views

Where a Spec Kit artifact is useful for interchange, review, or agent integration, Drydock generates it as a compatibility view over the authoritative Typed Specification and build state.

- `spec.md` is a generated feature-level compatibility view assembled from the owning typed Specification files. It is presented through the QuarterDeck or exported on demand.
- `plan.md` is a generated compatibility view over `ARCHITECTURE.md`, `DATABASE.md`, and the relevant planning state in `MANIFEST.md`.
- `tasks.md` is a generated compatibility view over execution objects, evidence, and review state already present in `MANIFEST.md` and the QuarterDeck.

These views improve compatibility without collapsing Drydock back into a single-file specification model.

Drydock adds capabilities with no Spec Kit equivalent:

Drydock capability	Description
`SCREEN-*.md`	Dedicated specification file type for UI screens
`drydock refit`	Governed Refit of the Blueprint and working software
QuarterDeck	Generated, throwaway review console; decisions write back into the build through the single decision writer
Agile plan mode	Spike-and-story delivery with per-object state and review gates
Ship's Log	Drydock-only JSONL append-only event ledger written automatically by development agents
Drydock Rigging	Technology governance propagated to all target projects
Ordered build planning	`BUILD_PLAN_COMPASS.md`-driven work ordering with context optimization
Brownfield import	Translate Spec Kit projects or source code into a Drydock Blueprint

Drydock capability	Description
Documentation generation	Blueprint-to-HTML documentation pipeline

Spec Kit Import Contract

```
drydock import <Target> <SpecKitProject> --format speckit
```

The translator reads `.specify/memory/constitution.md` and each Spec Kit feature directory, then creates a normal Drydock Blueprint. The resulting Drydock files become authoritative after product-owner review.

Spec Kit input	Drydock destination
`.specify/memory/constitution.md`	Project-specific intent, constraints, and success criteria in `COMPASS.md`; reusable engineering rules remain governed by Drydock
`specs/<feature>/spec.md`	One `FEATURE-{Name}.md`; clearly identified UI behavior also contributes to `SCREEN-*.md`
`spec.md` user stories and acceptance scenarios	Feature behavior and acceptance criteria in the owning `FEATURE-*.md`
`spec.md` success criteria and assumptions	`COMPASS.md` when project-wide; otherwise the owning `FEATURE-*.md`
`plan.md` technical context and structure	`ARCHITECTURE.md`, `METADATA.md`, and `DATABASE.md` where applicable
`research.md` accepted decisions	The owning `FEATURE-*.md`, `ARCHITECTURE.md`, or `DATABASE.md`
`research.md` unresolved decisions	`## Open Questions` in the owning Drydock file
`data-model.md`	`DATABASE.md`
`contracts/`	Routes and interfaces in `FEATURE-*.md` and `ARCHITECTURE.md`
`quickstart.md`	Useful operating instructions in `README.md` or `AGENTS.md`; otherwise ignored
`tasks.md`	Generated `tasks.md` compatibility view plus QuarterDeck task state projected from `MANIFEST.md`

Translation performs these steps:

1. Discover the Spec Kit constitution and feature directories.
2. Scaffold the standard Drydock Blueprint.
3. Classify project-wide intent, feature behavior, screens, architecture, persistence, and interfaces.

4. Merge each statement into its owning Drydock file.
5. Preserve unresolved or conflicting statements as open questions.
6. Generate relationship headers and validate the proposed Blueprint.
7. Write a conversion report listing mapped, duplicated, ambiguous, and ignored content.

The conversion report is review evidence, not a permanent Specification file. The translator must not silently discard ambiguous or conflicting source content.

Integration Behaviors

Initial useful behaviors:

Behavior	Drydock use
`clarify`	Resolve blocking open questions in the owning Specification files
`checklist`	Generate focused checks for a feature, screen, or ticket
`analyze`	Assist build-plan optimization and clarify uncertain implementation order
Agent integrations	Expose Drydock workflows to supported coding agents

Rules:

1. Spec Kit output must resolve back into the Drydock Blueprint.
2. Imported Spec Kit artifacts are inputs, not a second source of truth.
3. Spec Kit directories generated by an adapter are disposable.
4. Drydock remains usable without Spec Kit.

Sources

- GitHub Spec Kit
- Spec Kit documentation